Sistema de pedidos com descontos

Engenharia de Software - Design Patterns
Projeto didático em Python | Padrão escolhido: Strategy

Objetivo: analisar um condicional por tipo, representar um cenário com objetos e mensagens e documentar a decisão arquitetural. O projeto foi criado para esta atividade, pois não havia um projeto prévio disponível.

1. Contexto e regras

Uma empresa faz um pedido. O desconto depende do seu porte. Os percentuais são regras fictícias para demonstrar o conteúdo do notebook, e não condições comerciais reais.

Porte	Desconto	Total para R$ 5.000
Startup	10%	R$ 4.500,00
PME	15%	R$ 4.250,00
Enterprise	20%	R$ 4.000,00
Desconhecido	0%	R$ 5.000,00

Escopo: calcular o total de um pedido, comparar duas implementações e verificar a preservação do comportamento. Não inclui banco de dados, autenticação ou interface gráfica.

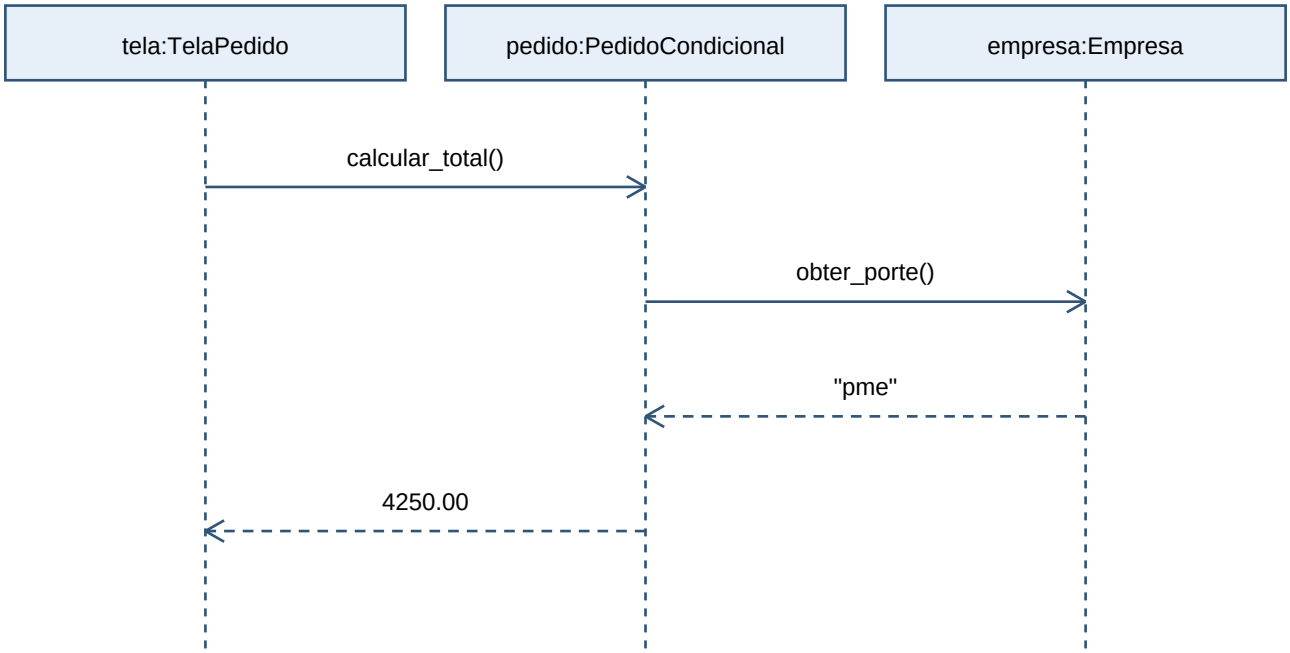
Valores monetários usam Decimal. Pedidos negativos e valores não finitos são rejeitados. O total é arredondado para duas casas com o modo padrão de Decimal (ROUND_HALF_EVEN).

2. Condicional e cenário inicial

O maior condicional por tipo está em **antes.py**, no método **PedidoCondicional.calcular_total()**: três ramos por porte e um caso padrão. A condição de validação do valor não é uma decisão por tipo.

```
porte = self.empresa.obter_porte()
if porte == "startup":
    desconto = self.valor * Decimal("0.10")
elif porte == "pme":
    desconto = self.valor * Decimal("0.15")
elif porte == "enterprise":
    desconto = self.valor * Decimal("0.20")
else:
    desconto = Decimal("0")
return (self.valor - desconto).quantize(Decimal("0.01"))
```

Cenário: empresa PME, pedido de R$ 5.000, desconto de R$ 750 e total de R$ 4.250. A tela chama o pedido; ele consulta o porte e executa o ramo correspondente internamente.



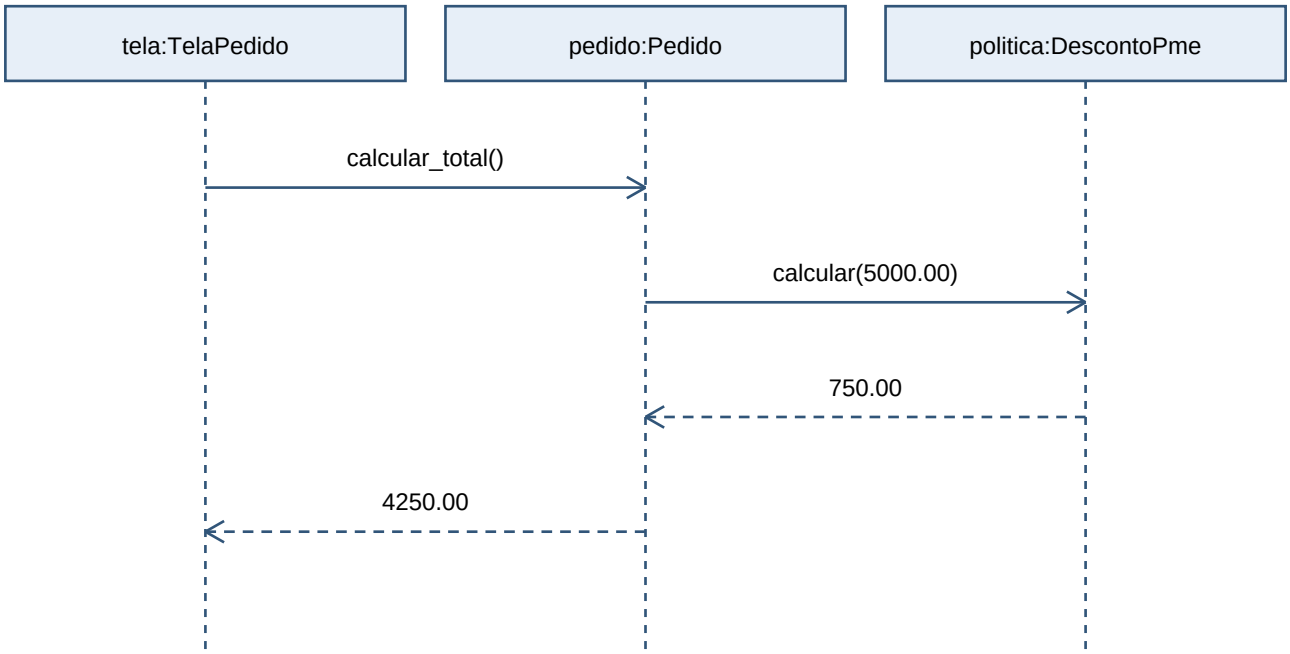
O diagrama representa apenas objetos e mensagens de um cenário concreto. O if ocorre dentro de calcular_total(); ele não é um objeto separado.

3. Escolha e cenário com Strategy

Variação em uma frase: Strategy absorve a variação das regras de desconto conforme o porte da empresa, permitindo alterar cada regra sem modificar o cálculo do total em Pedido.

PoliticaDesconto define calcular(valor). DescontoStartup, DescontoPme, DescontoEnterprise e SemDesconto implementam essa interface. Pedido recebe uma política no construtor e delega o cálculo do desconto a ela.

TelaPedido consulta o porte e usa selecionar_politica para configurar o Pedido antes de calcular. Esse seletor é uma fábrica simples, não uma implementação do Factory Method GoF baseado em especialização por subclasses.



Neste diagrama, o Pedido já recebeu DescontoPme. Ele não consulta o porte: calcula o total usando a política injetada. O resultado permanece R$ 4.250.

Adicionar uma categoria exige uma nova política e seu registro no seletor. A decisão por tipo continua concentrada nesse ponto; Strategy separa a execução das regras.

4. ADR-001 - Decisão arquitetural

Status: Aceito no projeto didático.

Contexto: Há três regras de desconto selecionadas por porte em um único método. Cada ramo desempenha a mesma responsabilidade com um percentual diferente.

Decisão: Usar Strategy, com uma interface comum e políticas injetadas no Pedido. Manter a seleção centralizada em selecionar_politica.

Alternativas: Manter o if seria razoável para estas regras pequenas e estáveis. Factory Method resolve criação; Observer resolve notificações; Adapter traduz interfaces; Decorator adiciona comportamentos por composição. Essas necessidades não são o foco deste cenário.

Consequências positivas: Cada política pode ser alterada e testada isoladamente. Pedido deixa de conhecer categorias específicas.

Consequências negativas: Aumenta o número de classes e exige manter um seletor. Para percentuais tão simples, um dicionário ou o condicional original também seriam suficientes.

Justificativa e limites: A adoção demonstra a refatoração estudada na aula. Não há histórico real de conflitos ou crescimento das regras que prove a necessidade em produção. A opção “nenhum ainda” seria defensável se o objetivo fosse somente resolver este cálculo mínimo.

Compatibilidade: As duas versões preservam percentuais, caso sem desconto e resultados. Testes de caracterização verificam a equivalência.

5. Execução e verificação

Requisito: Python 3. Não há dependências externas. Extraia o ZIP, abra a pasta projeto_descontos no VS Code e execute os comandos no terminal dessa pasta:

```
python main.py
python -m unittest -v
```

No Windows, use py no lugar de python se necessário. A demonstração imprime os resultados das duas versões lado a lado.

Verificação: testes cobrem os quatro portes, equivalência dos resultados, valor zero, normalização de porte com espaços e letras maiúsculas, e rejeição de valores negativos e não finitos.

Arquivos do projeto

modelos.py: Empresa e validação monetária.

antes.py: Implementação original com condicional.

strategy.py: Interface, políticas, seleção e Pedido refatorado.

main.py: TelaPedido e demonstração executável.

test_descontos.py: Testes com unittest.

docs/ADR-001.md: Registro editável da decisão.

docs/antes.mmd e depois.mmd: Fontes Mermaid dos diagramas.

Materiais utilizados

Leandro Ramos da Silva. *Aula 08 - Padrões GoF.pdf*, 2026: Strategy, sequência, refatoração e decisão arquitetural. *DesignPatterns.ipynb*: exemplos de políticas de desconto por porte. Ambos foram fornecidos pelo aluno como base da atividade.

O relatório e o código são uma elaboração didática própria. Os arquivos originais da aula não foram incluídos no pacote.